Introduction to Parsing

Team: Hazem Khaled · Eslam Elhousiny · Habiba Eid · Filopater Fayek · Farha Ashraf

Made with GAMMA

Parsing

is a fundamental process in compiler design. It is the process of analyzing a sequence of tokens from the source code to determine its grammatical structure according to the rules of a programming language.

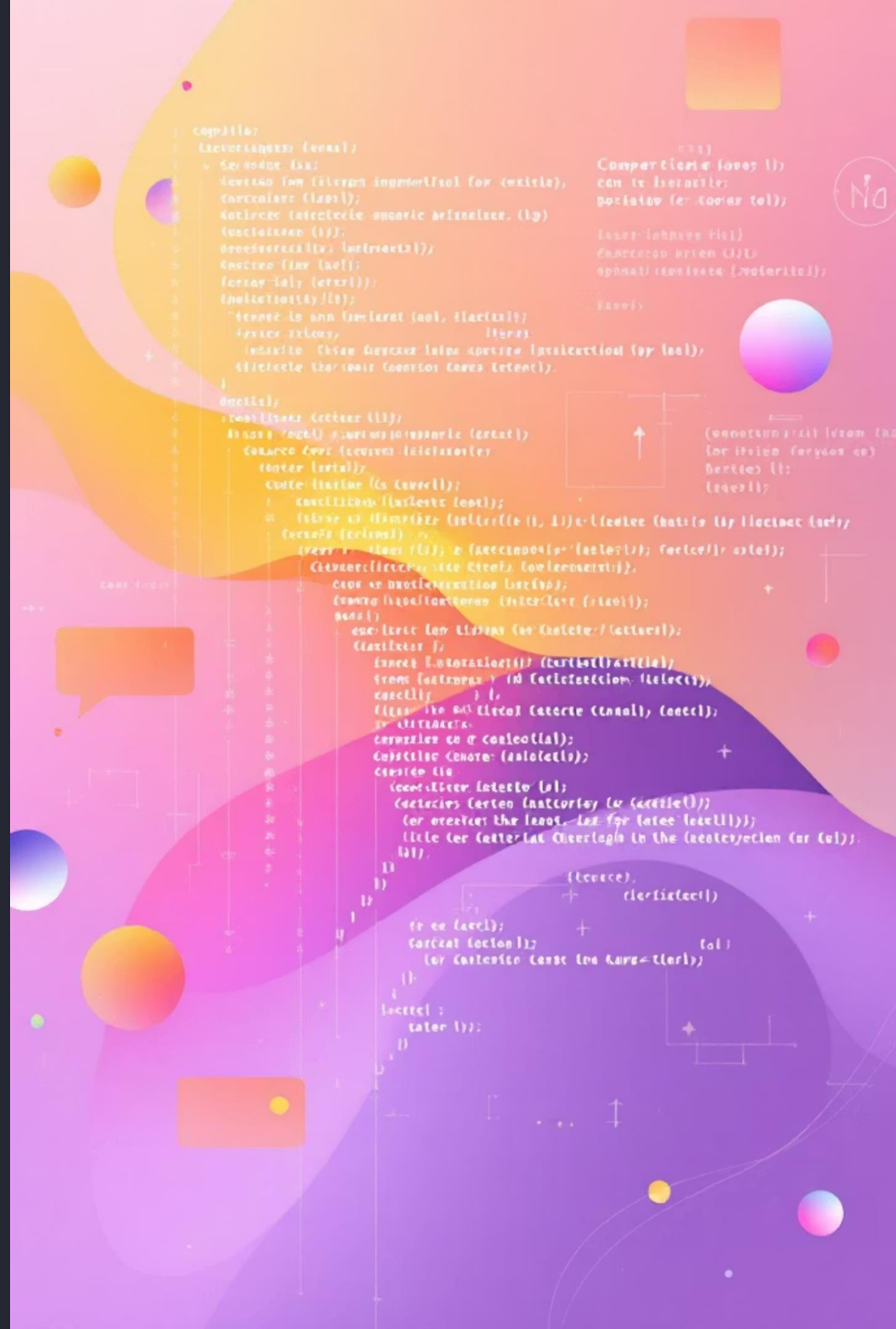
During parsing, the compiler checks whether the program follows the syntax rules defined by the language grammar. The result of this process is usually represented as a ***Parse Tree*** *or* ***Syntax Tree****, which shows the hierarchical structure of the program.*

Made with GAMMA

Why Parsing is Important

Parsing plays a crucial role in understanding and processing source code. It allows the compiler to verify that the code is written correctly according to the language grammar.

Through parsing, syntax errors can be detected early in the compilation process. It also organizes the structure of the program, making it easier for later stages of the compiler, such as semantic analysis and code generation, to process the code efficiently.



Types of Parsing

There are two main approaches used in parsing:

Top-Down Parsing

This approach starts from the start symbol of the grammar and attempts to derive the input string step by step.

Bottom-Up Parsing

This approach starts from the input string and tries to reduce it step by step until it reaches the start symbol.

Common parsing techniques include ***Recursive Descent Parsing, LL Parsing, and LR Parsing***

Made with GAMMA

Top-Down Parsing Overview

*Top-Down Parsing is a parsing technique used in compiler design where the parser starts from the **start symbol of the grammar** and tries to derive the input string step by step.*

*The parser expands the grammar rules in order to match the input tokens. This approach builds the **parse tree from the root to the leaves**, moving from the higher-level structure of the program to the detailed components.*

Top-Down parsing attempts to predict which production rule should be applied in order to generate the input string.

How Top-Down Parsing Works

01

Start Symbol

*In Top-Down Parsing, the parser begins with the **start symbol** and repeatedly applies grammar production rules to generate the input.*

02

Rule Selection

The parser checks the input tokens and chooses the appropriate rule that can produce the next part of the input string.

03

Completion

The process continues until the generated symbols match the entire input.

04

Implementation

*This method is often implemented using techniques such as **Recursive Descent Parsing** and **Predictive Parsing**.*

Made with GAMMA

Advantages and Limitations of Top-Down Parsing

Advantages

Top-Down parsing is considered simple and easy to implement compared to other parsing techniques.

It is widely used for building parsers manually because the logic is straightforward and closely follows the grammar rules.

Limitations

However, Top-Down parsing has some limitations. It cannot handle ***left-recursive grammars****, and some grammars must be modified before they can be parsed using this approach.*

Despite these limitations, it remains an important and commonly studied parsing technique in compiler design

Made with GAMMA

Recursive Descent Parsing

*Recursive Descent Parsing is a **top-down parsing technique** used in compiler design. It works by creating a set of **recursive functions**, where each function corresponds to a **non-terminal symbol** in the grammar.*

The parser starts from the start symbol and tries to match the input string by calling these functions recursively.

Each function processes a part of the input and checks whether it follows the grammar rules.

Made with GAMMA

How Recursive Descent Parsing Works

In Recursive Descent Parsing, the grammar rules are implemented directly as functions in a programming language.

*Each **non-terminal symbol** in the grammar has a corresponding function. These functions call each other according to the grammar production rules.*

The parser reads the input tokens one by one and checks if they match the expected symbols.

If the input follows the grammar rules, the parsing process continues until the entire input string is successfully analyzed.

Made with GAMMA

Advantages and Limitations

*Recursive Descent Parsing is simple and easy to understand. It is often used to implement **hand-written parsers** because the code structure closely follows the grammar.*

Another advantage is that it is flexible and easy to debug.

*However, this method has some limitations. It cannot handle **left recursion**, and some grammars must be modified before using this parsing technique.*

Despite these limitations, Recursive Descent Parsing is widely used for educational purposes and small language parsers.

Made with GAMMA

LL Parsing

*LL Parsing is a **top-down parsing technique** used in compiler design to analyze the structure of a program according to grammar rules.*

*The term **LL** stands for:*

- ***Left-to-right scanning of the input***
- ***Leftmost derivation of the sentence***

*This means the parser reads the input from left to right and constructs the parse tree by always expanding the **leftmost non-terminal** first.*

LL parsing is commonly used in predictive parsers.

Made with GAMMA

How LL Parsing Works

*LL parsers use a **parsing table** and a **lookahead token** to decide which production rule should be applied.*

*The most common type is **LL(1) parsing**, where the parser looks at **one input symbol at a time** to make parsing decisions.*

During parsing, the parser compares the current input symbol with the parsing table to determine the correct production rule.

This approach allows the parser to analyze the input efficiently without backtracking.

Made with GAMMA

Advantages and Limitations of LL Parsing

LL parsing is efficient and relatively simple to implement. It allows the parser to predict the correct production rule using a parsing table, which makes the parsing process faster.

Another advantage is that it does not require backtracking, which improves performance.

However, LL parsing cannot handle all types of grammars. Some grammars must be modified, such as removing ***left recursion*** *and performing* ***left factoring****, before they can be used with LL parsers*

Made with GAMMA

LR Parsing

*LR Parsing is a **bottom-up parsing technique** used in compiler design to analyze the syntactic structure of a program.*

*The term **LR** stands for:*

- ***Left-to-right scanning of the input***
- ***Rightmost derivation in reverse***

*Unlike top-down parsing, LR parsing starts from the **input symbols** and gradually reduces them until it reaches the **start symbol** of the grammar.*

This method is widely used in modern compiler construction because it can handle a larger class of grammars.

Made with GAMMA

How LR Parsing Works

*LR parsers use a **stack-based mechanism** to process input symbols.*

The parser performs two main operations:

***Shift** The parser reads the next input symbol and pushes it onto the stack.*

***Reduce** The parser replaces a sequence of symbols on the stack with a non-terminal symbol according to a grammar rule.*

By repeatedly applying shift and reduce operations, the parser eventually reduces the entire input into the start symbol.

Made with GAMMA

Types and Advantages of LR Parsing

There are several types of LR parsers used in compiler design, including:

- ***SLR (Simple LR)***
- ***CLR (Canonical LR)***
- ***LALR (Look-Ahead LR)***

*Among these, **LALR parsers** are the most commonly used in practical compiler tools.*

LR parsing is powerful because it can handle a wide range of programming language grammars. It is also more reliable than many top-down parsing techniques.

However, LR parsers are more complex to design and implement compared to simpler parsing methods.

Made with GAMMA

Thank You

Supervised by:

Dr. Noha El-Shoafy